



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRACTICA N°3

MODELADO GEOMÉTRICO

NOMBRE COMPLETO: García Cruz Andrea

N.º de Cuenta: 316172697

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 01

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 17/09/26

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

Edición de Código

La edición se realizó principalmente sobre el código base proporcionado para la práctica. En lugar de construir todas las geometrías desde cero, se modificaron valores de transformación y se incorporaron variables para facilitar los ajustes. Esta estrategia permitió cambiar el tamaño de una pieza y recalculat la posición de las piezas que dependían de ella.

Un ejemplo importante es el brazo. Se utilizó la variable “largobrazo1” para controlar su longitud y, posteriormente, se empleó esa misma variable para calcular la posición de la segunda articulación. Esto evita que al cambiar la longitud del brazo aparezca un espacio entre las piezas.

```
439 //ARTICULACION DEL BRAZO #1
440 model = modelaux;
441 float largobrazo1 = 5.0f;
442 model = glm::translate(model, glm::vec3(-1.5f, 1.5f, 0.0f)); //flota o no
443 model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f)); //rotacion Z 45°
444 model = glm::translate(model, glm::vec3(0.0f, 0.5f, 0.0f));
445 modelaux = model; //Pivote para el siguiente eslabon
446 model = glm::scale(model, glm::vec3(0.5f, largobrazo1, 0.5f)); //MITAD DEL CUBO
447 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
448 color = glm::vec3(1.0f, 0.0f, 0.0f); //ROJO
449 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
450 meshList[2]->RenderMeshGeometry();
```

Después, la siguiente articulación toma como referencia la matriz auxiliar y utiliza la mitad de la longitud del brazo para posicionarse:

```
452 //articulación 2
453 model = modelaux;
454 model = glm::translate(model, glm::vec3(0.0f, largobrazo1 / 2.0f, 0.0f));
455 modelaux = model;
456 model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
457 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
458 color = glm::vec3(0.5f, 0.5f, 0.5f); //ejes del mismo color
459 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //cambi de color
460 sp.render();
```

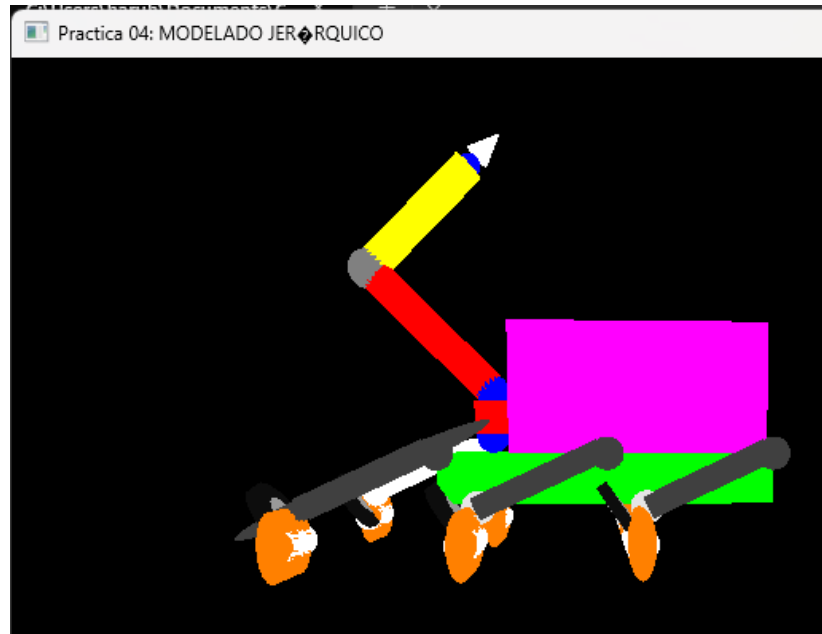
También se ajustó la generación del cono. En el código se genera primero el vértice superior y después los puntos de la circunferencia de la base mediante funciones seno y coseno. Se añadió el punto correspondiente al inicio de la circunferencia para cerrar correctamente la base.

```
239 vertices.push_back(R * cos(0));
240 vertices.push_back(-0.5);
241 vertices.push_back(R * sin(0));
```

Otro aspecto relevante fue el uso de matrices auxiliares. La variable modelaux conserva un punto de referencia para conectar una pieza con la siguiente, mientras que modelaux2 permite regresar a la base del rover para construir ramas independientes, como las correspondientes a las ruedas.

Ejecución de Código

Al ejecutar el programa se genera una ventana de OpenGL en la que se visualiza el modelo tridimensional construido a partir de las primitivas definidas en el código. La configuración inicial crea una ventana de 800×600 píxeles, inicializa la cámara, carga los shaders y prepara las geometrías.



Problemas presentados

- ❖ Piezas separadas de su punto de unión: Al modificar la longitud de un segmento, la pieza siguiente no se desplazaba junto con él y quedaba un hueco visible. Esto se debía a que la posición de la siguiente articulación estaba calculada con el tamaño original.
- ❖ Rotaciones no heredadas: En algunos casos, el punto de referencia se guardaba antes de aplicar una rotación. Como consecuencia, la siguiente pieza no tomaba correctamente la orientación de la pieza anterior.
- ❖ Forma incorrecta del cono: El cono podía aparecer deformado debido al cálculo incorrecto de uno de los vértices que cerraba la base circular.
- ❖ Ajustes de tamaño sin efecto aparente: Al cambiar algunas dimensiones, el resultado visual no correspondía con el valor modificado porque la dimensión dependía de otra transformación relacionada con la posición.
- ❖ Para solucionar estos problemas fue necesario revisar el orden de las transformaciones “traslación, rotación y escalado” y verificar que cada pieza utilizara como referencia la matriz correspondiente al elemento del que dependía.

Conclusión

Esta práctica me permitió comprender mejor que el modelado jerárquico no consiste únicamente en dibujar varias figuras 3D, sino en establecer correctamente la relación entre ellas. Al trabajar con el rover pude observar que un cambio pequeño en una pieza puede afectar a todas las que dependen de ella, por lo que es importante guardar los puntos de referencia en el momento adecuado y utilizar matrices auxiliares para conservar las transformaciones necesarias.

También aprendí que muchos de los errores que aparecen en pantalla se deben al orden de las transformaciones o a utilizar una referencia incorrecta. La revisión paso a paso y el uso de variables para controlar las dimensiones facilitaron los ajustes del brazo y de sus articulaciones. En general, el ejercicio reforzó mi comprensión de las transformaciones geométricas y de cómo utilizarlas para construir modelos 3D compuestos y articulados.

Bibliografía en formato APA

- Anthropic. (2026). Claude (Sonnet 5)[Modelo de lenguajes de gran escala]. <https://claude.ai/>
- Universidad Nacional Autónoma de México, Facultad de Ingeniería. (2026). Ejercicios de clase N.º 4: Modelado jerárquico. Laboratorio de Computación Gráfica e Interacción Humano-Computadora.
- García Cruz, A. (2026). Práctica 4: Modelado jerárquico [Código fuente en C++ con OpenGL/GLM].